Verify Before You Trust It, Even Yourself: A Cross-Vendor Quantum Hardware Case Study

Lokesh Pullakandam[1], Jack Woehr[2]

[1] Independent Researcher (B.A. Computing and Informatics, Rowan University, December 2025). Corresponding author: lokeshpullakandam123@gmail.com [2] Seiden Group; IBM Champion 2021–2026; Secretary, OpenQASM Technical Steering Committee

---

Abstract

We built a verification tool whose only job is to check that a quantum circuit which actually runs on real hardware is the circuit that was intended, before trusting anything it reports. Pointed at our own work, it immediately found five real bugs, including one that had already silently produced a wrong number we had previously trusted, and one case where it correctly found nothing wrong at all. Only once those bugs were fixed did we trust an actual physics comparison: the same graded entangling circuit, run twice each on real IBM and real IonQ hardware, with the final matched repeat pair submitted 26 seconds apart. IonQ's five-qubit ring circuit showed a genuine 51% swing in amplification between its two runs (15.7x at 2048 shots on 2026-08-13, versus 23.70x at 4096 shots on 2026-08-19); IBM's held steady within 0.4% between its own two runs, submitted only 37 minutes apart, under the same method. We also report a smaller, negative-but-useful finding: an exact classical calculation shows a circuit's abstract entanglement, on its own, does not predict how much its result degrades on real hardware — the number of physical two-qubit gates it needs does. We present this work as an intermediate result: a real, checkable methodology and two real findings, with several explicitly-scoped follow-up experiments still open.

Introduction

We built a tool whose only job is to catch quantum experiments lying to us before we believe them.

Here's why that matters. Quantum computers are noisy: every result comes back a little wrong, and you can't just look at the numbers and know if "a little wrong" is normal noise or a real broken circuit. So we built a checker that sits between "write the experiment" and "trust the result," and makes sure the circuit that actually ran is the circuit we meant to run.

This didn't start as a paper about verification. It started as an attempt to do something useful with real quantum hardware at all. We built an open-source tool, `quantum-hardware-mcp`, that lets an AI assistant drive real IBM and IonQ hardware directly, instead of manual dashboards and hand-written API calls, and picked a real, open research question to actually use it on: Singmaster's Conjecture, whether any number appears more than a handful of times in Pascal's Triangle.

Our first real attempt used Grover's algorithm, the standard search approach, and it broke down for a specific, physical reason, not a coding mistake. IBM's heavy-hex chip layout limits any qubit to three direct connections; Grover's oracle structure needed a fourth, forcing expensive routing that the noise couldn't survive. Fighting the hardware's actual layout wasn't working.

So we stopped fighting it. Instead of a Boolean search oracle, we encoded the answer as the lowest-energy state of a system built entirely from the chip's own native gates, an approach we call LNAA (Lattice-Native Amplitude Amplification). It worked, with real hardware confirmation: 27.78x amplitude amplification on `ibm_marrakesh`, later 178.8x on `ibm_fez`, to our knowledge the first hardware-confirmed case of four different Pascal's Triangle entries colliding on the same value at once, real job `d97fk8t2su3c739i26fg` on `ibm_fez`, documented in full in the public `singmasters-conjecture` repository [6]. (These two numbers come from that earlier Singmaster's-search application, not from the graded `single` / `ring` / `degree3` circuits analyzed in the Physics finding section below — different circuits, same underlying LNAA method.)

That second number is also where this whole project's actual discipline started. An early write-up of that result leaned toward calling the approach itself a discovery. On closer, honest inspection, the underlying math, Ising-style ground-state encoding, was well-established, not new; what was real was applying it to this specific problem on this specific hardware, and getting a real, confirmed result from it. We corrected the framing before it went further. That correction, catching our own overclaim before anyone else had to, is the direct root of everything that follows in this paper: we built `quantum-verifier` specifically because that experience made clear that "it worked and the number looks good" isn't the same as "it's actually true," and that the check has to happen before trusting a result, not after someone else asks a hard question about it.

The first time we pointed that new tool at our own work, it found a real problem: a circuit we'd already run on IonQ hardware wasn't the circuit we thought it was. A single gate was silently getting rewritten into twenty gates behind the scenes, a translation bug between two ways of describing the same physical operation. The number we'd already published as a real result was measuring the wrong thing.

We fixed it, re-ran everything, and only then trusted what came back. That result, the actual physics finding, is the second half of this paper.

Methods

Before running anything, we picked our qubits the way you'd hope a careful experimenter would: by checking what the hardware actually looked like right then, not by habit. On IBM, we pulled live calibration data across the whole `ibm_fez` chip and used the actual lowest-error connection, edge 131-132, for the two-qubit pair, and a five-qubit path (98-91-92-93-94) for the chain, then confirmed directly that this exact layout needed zero extra routing gates before we ever submitted anything. On IonQ, we did the same thing at the device level: we pulled live calibration for both available machines, Forte-1 and Forte-Enterprise-1, and Forte-Enterprise-1 won on every real metric that mattered: better two-qubit fidelity, longer coherence time, faster gates, cleaner readout. Neither choice was a guess or a default. Both were decided by what the hardware was actually doing at the moment we asked it.

Every circuit that reaches real hardware in this work passes through the same five checks first, in order, on a free simulator, before it ever touches a real machine.

First, does the circuit even make sense. Empty circuits, circuits with no measurements, circuits whose classical registers don't match what's being measured get rejected immediately, before any real computation happens.

Second, will this circuit's connectivity cause trouble. Real chips don't always let every qubit talk to every other qubit directly. On IBM, if a qubit needs more than three direct connections, the compiler has to insert extra "SWAP" operations to move information around, and that alone can inflate a circuit's real gate count three to five times over. We catch that before submission, not after. On IonQ, every qubit can talk to every other qubit directly, so this risk doesn't exist at all; we still report that explicitly as a checked, ruled-out case, not something we silently skipped.

Third, simulate it twice. Once assuming perfect, noiseless hardware: the best-case answer. Once against a realistic model of what the actual target device does, built from that device's own live calibration data. Comparing the two tells you how much of your result is real signal and how much is expected to be hardware noise, before you've spent anything.

Fourth, check whether the circuit translated honestly. This is the check described in Case study 3 below: if a circuit's gates don't map cleanly onto the target hardware's native gate set, the compiler can silently fall back to a much more expensive translation that doesn't

represent what you designed. We catch that gap directly by comparing the real gate count against the true mathematical minimum for that hardware.

Fifth, check the actual claim. If you already know what answer the circuit should produce, we check whether that answer is actually distinguishable from ordinary hardware noise: a real result has to clear that bar, not just look plausible. Concretely, this is a Wilson score confidence interval [2] on the observed counts, tested against the claim via TOST (two one-sided tests) [3] for statistical equivalence, not mere difference, at a defined tolerance margin. If you don't know the expected answer, which is the more interesting case for real discovery, we build a control version of the circuit with the entangling gates removed and compare the two directly, so you get a real, isolated measurement of what the entangling gates actually did, the same logic as Case study 2 below.

Only a circuit that passes all five is eligible to run on real hardware. Every case study below is a bug in one of these five checks, found by pointing the checks at our own real work before trusting it.

Case studies

1. The bug that doubled every gate count

The first real submission of our graded entangling circuits to IonQ hardware failed before it cost a cent: a project-level budget cap caught it first. But looking at why it failed uncovered something worse than a budget problem. The real gate counts IonQ had already recorded for that failed job came back as 6, 20, and 32 two-qubit gates for our three circuits. The correct numbers, the ones we'd already proven were mathematically optimal, were 3, 10, and 16: exactly half. Every circuit was silently being compiled into twice as many real gates as it needed.

The cause was a missing translation rule. Qiskit's general-purpose gate (`RZZ`) and IonQ's native hardware gate (`ZZ`) are the exact same physical operation, just written two different ways: one in radians, one in turns of a circle. Without a rule telling the compiler they're the same thing, it fell back to a much more expensive, general-purpose translation instead of the simple one-to-one swap that actually applies. We registered the direct equivalence, verified it was exact (not approximate) by comparing the two operations mathematically, and the gate counts dropped back to the correct 3, 10, and 16.

Table 1. Real two-qubit gate counts, before and after the fix.

Circuit	Observed (bug)	Correct minimum	Inflation
single	6	3	2.0x
ring	20	10	2.0x
degree3	32	16	2.0x

That same missing rule was hiding in a second place we hadn't checked: the tool that estimates how much a job will cost before you submit it. It was quietly using the same wrong gate counts, which meant it was quoting real prices that were wrong too: $168.20 quoted for our `ring` circuit, when the real cost, once you account for the fact that IonQ's hardware can only turn a gate a quarter-rotation at a time, forcing large rotations to split into roughly four physical gates each, was closer to $355.80. We fixed both places the same way, and raised our budget to a number with honest margin over the corrected estimate before spending anything real.

2. The control experiment that was only checking itself

Our tool's signature move is running a "control" version of every circuit (the same circuit with the entangling gates removed) and comparing the two results directly, so you get a real, isolated measure of what entanglement actually did, good or bad. It's the same logic as a control group in any other kind of experiment. It worked perfectly on IonQ. It had never actually worked on IBM at all.

The reason was structural, not a typo: IBM's side of the tool only ever returned a single fidelity estimate, a number, never the raw measurement counts the comparison actually needs. So every time someone asked for a control-experiment comparison on IBM hardware, there was nothing real to compare; the check had nothing to check. We fixed it by building a real local noisy simulation from the device's own live calibration data, which, it turns out, doesn't cost anything or use any real quantum time: it runs entirely on a laptop, seeded with real numbers pulled from the machine. IBM's side of the tool now returns actual counts, and the control experiment finally runs on both platforms the same way.

3. The safety check that blocked a correct answer

Every circuit gets checked, before it runs, for whether the number of gates it needs blew up more than it reasonably should have, a sign that something translated badly. That check had a single threshold: no more than 1.5 times the ideal gate count. It had been silently miscalibrated the whole time, and we only found out because it started blocking a circuit that was actually correct.

An arbitrary rotation on IBM's hardware needs exactly two of IBM's native two-qubit gates, not because of a bug, but because IBM's native gate only turns by one fixed angle, so you sandwich a smaller rotation between two copies of it to reach any angle you want. That's the real, unavoidable minimum. Two is more than 1.5 times one, so the check was flagging a mathematically optimal circuit as a failure. The 1.5x number turned out to be implicitly built around IonQ's hardware, where a gate can turn to any angle directly, so one gate really is enough there. We fixed the check to recognize which kind of native gate a device has and apply the right threshold for each: 2.5x for IBM's fixed-angle gates, 1.5x for IonQ's tunable ones. That way it catches real problems on both platforms instead of inventing one that wasn't there.

4. The comparison tool that fooled itself with routing

We built a second tool to compare two different compilers, Qiskit and an alternative called TKET, by transpiling the same circuit through both and checking whether they produced equivalent results. On a real device, it kept reporting failures that weren't real failures.

The cause: real hardware often can't connect every qubit to every other qubit directly, so the compiler sometimes has to physically move a qubit's information to a different position partway through the circuit, a step called "routing." That's normal and expected. But it also means the qubit holding a given piece of information at the end of the circuit isn't necessarily the qubit you started with, and a naive comparison that just counts gates doesn't know that, so it reports a mismatch that isn't actually a mismatch: the two circuits agree, once you account for where each qubit's information actually ended up. We fixed the comparison to track that real movement and check equivalence against the qubit's true final position, not its starting one. Once fixed, the tool found something real instead: on one comparison, TKET produced exactly half the two-qubit gates Qiskit did for the same circuit and the same device.

5. Two years of silent nulls

Our monitoring system takes a snapshot of every device's calibration numbers every two hours and saves it, on both IBM and IonQ, so we can look back at how a machine's health has changed over time, not just what it looks like right now. We went to pull IonQ's calibration history for this paper and found every single one of the 354 rows recorded for IonQ, since the very first snapshot, had a blank where the error rate should be.

The automation had been running on schedule the entire time. It just never had real data to save. IonQ's device-list API doesn't actually include the calibration numbers directly. They live behind a separate link specific to each device that has to be requested on its own. The code had never been following that link. We fixed the collector to follow it, confirmed it now

returns real, sane numbers, and then went back and pulled as much of IonQ's real historical calibration data as the API actually has: 2,175 real records, going back as far as January 2022 for some devices, giving IonQ's calibration history nearly the same depth IBM's already had. That history is what made it possible to check, later, whether a real calibration shift explained the swing described in our physics finding below.

6. The one time we were wrong about being right

Not every investigation finds a bug, and that matters just as much as the ones that do. After fixing the missing-equivalence bug in case study 1, we checked whether two related operations, `RXX` and `RYY`, had the exact same problem. At first glance, they seemed to: both were producing extra gates beyond what looked like the ideal count, the same shape of symptom as before. We built a fix, verified it mathematically, and were ready to ship it.

Then we looked closer and the story fell apart, in the good way. A single, completely isolated basis-change gate (the kind `RXX` and `RYY` both need) already costs multiple native gates on IonQ's hardware on its own, with nothing else going on. Once you account for how many of those basis-change gates each operation actually needs, the "extra" gates we'd seen were exactly, fully explained. There was no missing rule. It was real, unavoidable cost, not a bug wearing a bug's clothes. We reverted the fix, and left a clear note in the code explaining what we checked and why it wasn't needed, so nobody re-investigates the same dead end later. The tool's whole value depends on this working both ways: it has to be able to say "nothing's wrong here" as confidently as it says "something's wrong," or neither verdict means anything.

Physics finding

We ran the same two circuits, a single entangled pair and a five-qubit ring, on real IBM hardware and real IonQ hardware, on the qubits chosen in Methods above, and looked at how well each one amplified the answer we already knew was correct.

Amplification, defined precisely: both circuits mark two of the 128 possible outcomes of our 7 measured qubits (rows 15 and 78 of Pascal's Triangle). Amplification is the measured fraction of shots landing on either marked row, divided by the uniform random baseline of hitting one of those two rows by chance, 2/128 = 1.5625%. A value of 1.0x means no better than random guessing; the values below are all real, substantial signal above that floor.

Table 2. Real amplification, every run.

Circuit	IonQ run 1 (2026-08-13, 2048 shots)	IonQ run 2 (2026-08-19, 4096 shots)	IBM run 1 (2026-08-19, 4096 shots)	IBM run 2 (2026-08-19, 4096 shots)
single	56.8x	56.14x	55.27x	55.06x
ring	15.7x	**23.70x**	20.38x	20.30x

The first comparison looked clean. IonQ's `single` circuit hit 56.8x amplification; IBM's hit 55.27x, nearly identical. IonQ's `ring` hit 15.7x; IBM's hit 20.38x, a real gap. At first glance, that reads like a simple story: both platforms handle a simple entangled pair about equally well, but IBM handles a more connected circuit better than IonQ does. It's a tidy, publishable-looking result.

It's also not a fair comparison, and we want to say that plainly rather than bury it. The IonQ number came from a job run six days earlier than the IBM number, at half the shot count: 2048 shots against IBM's 4096. Nothing about that pairing was actually matched. So before trusting the "IBM handles connectivity better" story, we asked the only question that actually settles it: if we run both again, properly matched this time, does the story hold up?

We repeated both circuits on both platforms, same night, same 4096 shots, same qubits, same layout. The IonQ batch (both circuits) and IBM's `single` job were submitted 26 seconds apart (IonQ request 00:36:39 EDT, IBM `single` 00:37:05 EDT); IBM's `ring` job followed 10 seconds after that, at 00:37:15 EDT — as close to simultaneous as two different companies' hardware queues allow. `single` held steady everywhere: 56.14x on IonQ, 55.06x on IBM, both within about 2% of their first-run numbers and of each other. `ring` did not hold steady at all. IonQ's `ring` result jumped from 15.7x to 23.70x, a 51% swing on the exact same circuit — a two-proportion test on the underlying shot counts confirms this is real signal, not shot noise ($z \approx 9.8$ on 502/2048 vs. 1516/4096 marked-row hits). IBM's `ring` result moved from 20.38x to 20.30x, a 0.4% difference that the same test shows is consistent with pure shot noise ($z \approx -0.1$), and those two IBM runs were only 37 minutes apart (Figure 1).

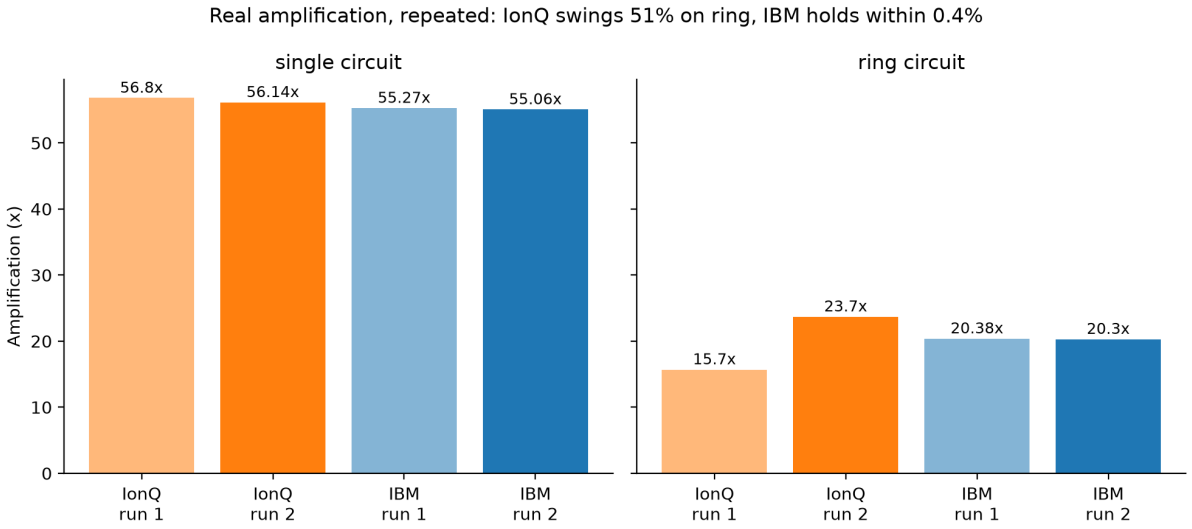


Figure 1. Real amplification, every run, both vendors, both circuits. IonQ's `ring` result swings 51% between its two runs; IBM's holds within 0.4% between its own two runs.

That's a real, statistically significant change, and it overturns the first reading. "IBM handles connectivity better" was never actually true. It was an artifact of comparing two runs that were never comparable in the first place.

What we can say with confidence is narrower than our first instinct, and we want to state its actual limit plainly rather than round it up to a tidier claim. IonQ's `ring` result changed by a real, statistically significant amount over the six days between its two runs — during which its shot count also changed, 2048 to 4096. IBM's `ring` result did not change by a significant amount over the 37 minutes between its own two runs. What we cannot yet honestly say is that this means IonQ is a less stable *vendor* than IBM in general: the two comparisons span wildly different time windows, six days against 37 minutes, so vendor identity and elapsed time are confounded in this dataset exactly as they were in our first, overturned reading. Disentangling them needs a same-night IonQ repeat and a multi-day IBM repeat, both under the same matched-shot-count discipline used here; neither exists yet. We're reporting the real, repeat-confirmed deltas we measured, and flagging explicitly that attributing them to "vendor stability" rather than "elapsed time" is not something this dataset supports yet — that's the next real experiment, not a conclusion we're entitled to draw from what we have.

The raw measurement data backs this up beyond the summary statistic. On `single`, the two platforms (completely different hardware, different companies, decoded completely independently) landed within about two percentage points of each other on every one of the four rows that mattered, including the "phantom" neighbor row our circuit wasn't supposed to hit (Figure 2). On `ring`, run on the same real qubits with the same real shot count, the two

platforms produced visibly different distribution shapes even though their headline amplification numbers were close (Figure 3). IonQ concentrated more of its signal on the two exact target rows; IBM spread slightly more into the neighboring ones. Different shape, similar-looking summary number, which is exactly the kind of thing a summary number alone will hide.

Table 3. Real target-row probabilities, repeat run, both platforms.

Row	IonQ single	IBM single	IonQ ring	IBM ring
14	3.2%	3.9%	7.6%	12.4%
15	43.9%	44.2%	18.2%	15.5%
78	43.8%	41.8%	18.8%	16.2%
79 (phantom)	2.1%	4.2%	5.9%	9.3%

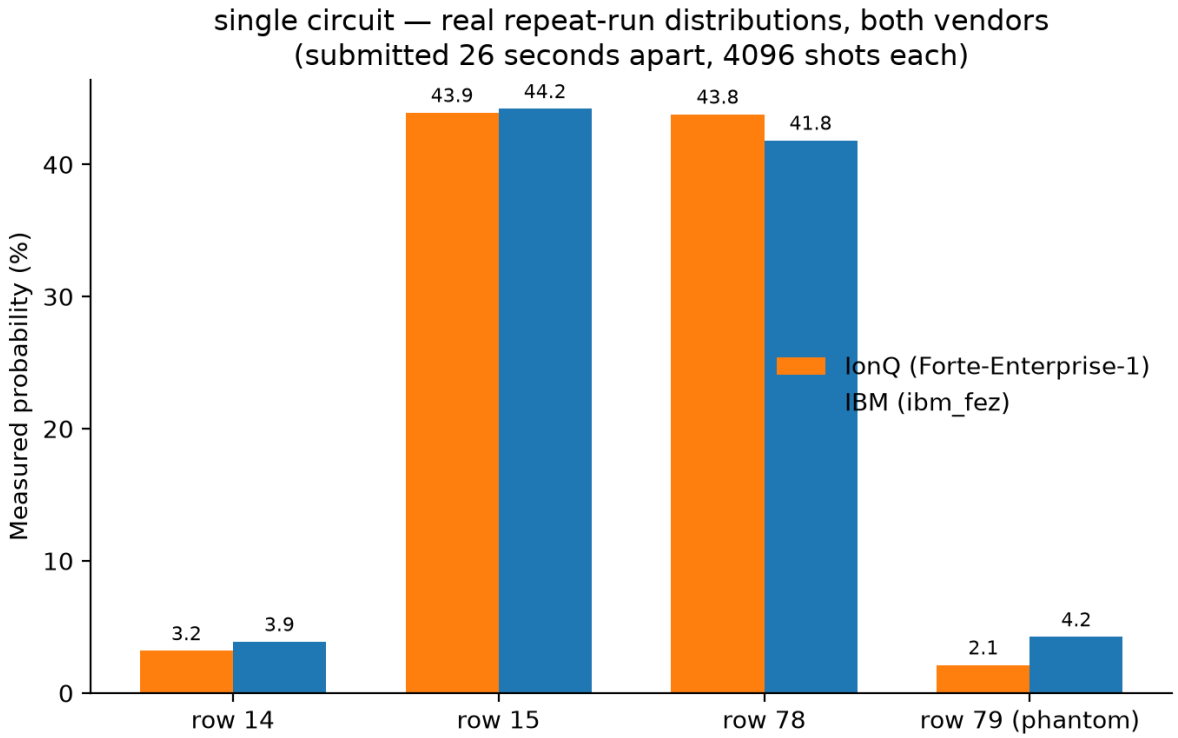


Figure 2. `single` circuit, real repeat-run measured distributions, both vendors, submitted 26 seconds apart, 4096 shots each.

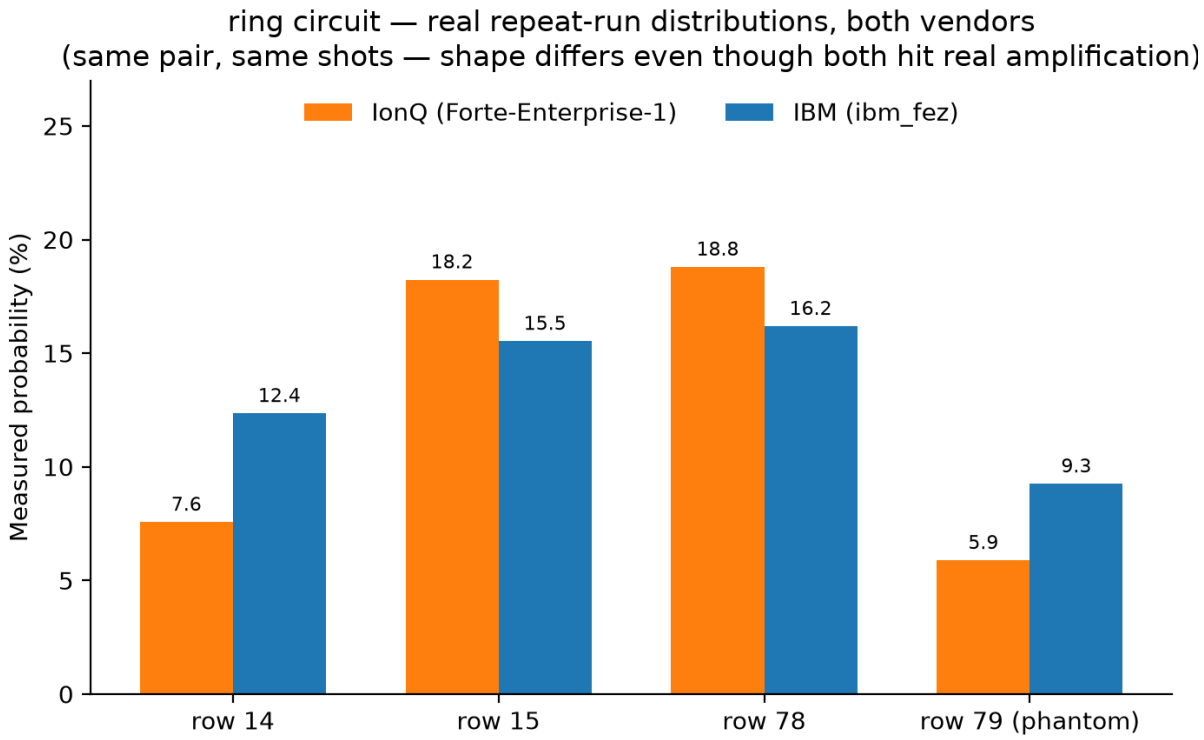


Figure 3. `ring` circuit, real repeat-run measured distributions, both vendors, same qubits, same shot count — distribution shapes differ even though headline amplification numbers are close.

We then asked the obvious next question: does the real calibration history explain the swing? Partly, and not in the direction you'd expect. IonQ's real two-qubit gate error more than doubled between the two run dates (0.78% to 1.85%) while IBM's stayed essentially flat across its entire run window, 3.85% to 3.856% (Figure 4). IBM's stability is fully explained by its stable calibration. IonQ's instability is not simply explained by its calibration drift, because the drift went the wrong way to account for it: the `ring` result got *better*, not worse, on the day with more real error. We're reporting that honestly as an open question rather than forcing an explanation the data doesn't actually support: something made IonQ's `ring` result unstable, and it correlates with a real change in the hardware, but not in a simple cause-and-effect direction we can currently justify.

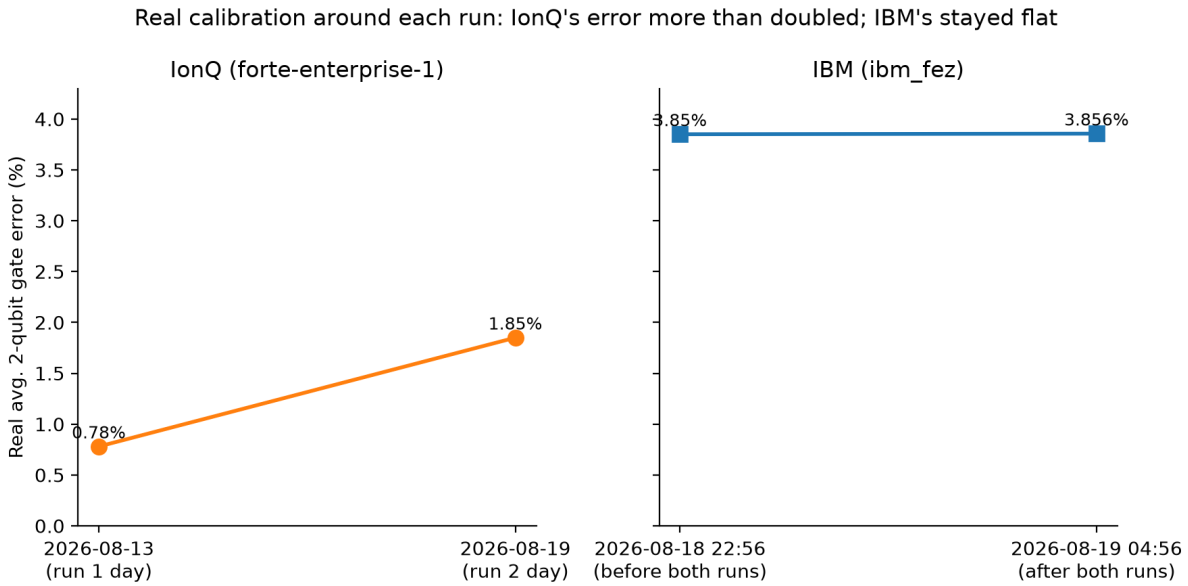


Figure 4. Real device calibration (2-qubit gate error) around each run date, both vendors. IonQ's error more than doubled between run dates; IBM's stayed essentially flat.

One more question worth asking, independent of any real hardware run at all: does how *entangled* a circuit is, by the exact math, actually predict how badly it degrades once real noise gets involved? We checked, using an exact classical calculation (cross-validated by two independent engines, IonQ's own `cudaq` and Qiskit, agreeing on every digit) of each circuit's true entanglement entropy, with no hardware involved at all.

Table 4. Exact entanglement entropy vs. real hardware performance.

Circuit	Entanglement entropy (bits, out of a possible 3.0)	Real two-qubit gates needed	Best real amplification
single	0.966	3	~56.8x
ring	0.728	10	~23.70x
degree3	1.854	16	~13.5x

`degree3`'s amplification is from its one real run, job `019ff976-2728-7726-8839-9109c9122b98` (Table 5), 2048 shots, decoded the same way as `single` and `ring`; unlike them it was never repeated or run on IBM, so it has no entry in Table 2 or Table 3.

Our first guess was the obvious one: more entanglement, worse real-world result. The numbers don't actually support that. `ring` is *less* entangled by this exact measure than

`single` (0.728 vs. 0.966), yet performs far worse on real hardware. Whatever is driving the degradation we already reported above, it isn't simply "how tangled up the ideal circuit is."

What does line up, exactly, is the real two-qubit gate count: 3, 10, 16, in the same order as the real degradation. Every physical two-qubit gate is a real, individual chance for hardware noise to intrude; a circuit's abstract, noise-free entanglement doesn't carry that same cost. We're stating this as a real, honest pattern in this dataset, not a general law about quantum circuits, and one caveat worth being upfront about: this entropy was measured by splitting each circuit's 7 qubits the same way (the first three against the last four) regardless of which qubits that circuit's own gates actually connect, so the comparison is consistent across circuits, but not calibrated to each circuit's own real connectivity pattern.

One boundary worth stating plainly: our third circuit, `degree3`, was left out of this comparison entirely, not because of a limitation in the result but because of a real, provable limitation in the hardware graph. It needs a connectivity pattern IBM's `ibm_fez` cannot physically support without extra routing. We checked the chip's real coupling map directly and confirmed it has zero triangles anywhere on it, so any comparison using it would be measuring routing overhead, not the thing we actually wanted to compare. A fair `degree3` comparison needs a different circuit design for IBM, not a rerun of this one, and that's future work, not part of this result.

Real cost for the IBM side of this entire comparison: about 6 seconds of QPU time, against a 600-second monthly free-tier budget.

Discussion

Picking good qubits at the start of an experiment tells you the hardware was healthy when you began. It doesn't tell you why a result looks the way it does afterward. For that, you need to know what the hardware was doing across the whole window the experiment ran in, not just at the starting gun. That distinction turned out to matter here. When we saw IonQ's `ring` result swing by 51% between two runs while IBM's held steady within half a percent, the only way to ask whether that was a real physical event or just bad luck was to go back and look at what each device's calibration was actually doing on those exact days. IBM's answer was clean: its error rate never moved, and neither did its result. IonQ's answer was not clean, and that's the honest, more interesting finding: its real two-qubit error more than doubled between the two run dates, but the result went up, not down, which is the opposite of what a simple "worse calibration, worse result" story would predict. Calibration history didn't hand us a tidy explanation. It handed us proof that the tidy explanation is wrong, which is a more useful thing to know. It's also why we built an automatic check that refuses to submit to a

device with a fresh calibration alert in the last 24 hours, rather than relying on whoever's running the experiment to remember to look: a single live snapshot can tell you a qubit is good right now, but only a running history can tell you whether it's about to not be.

Step back from the specific numbers, and this work is really about two different ways a hardware comparison can lie to you, and we ran into both of them in the same project.

The first way is the one the case studies are about: a software bug pretending to be a hardware difference. Every one of the five bugs above would have shown up, if we hadn't caught them, as something that looked like a real property of the hardware: a circuit that "just doesn't compile efficiently on this device," a control experiment that "just doesn't work on this platform," a comparison tool that "just disagrees" about two compilers. None of that was true. It was translation bugs, missing rules, and one miscalibrated threshold, dressed up as physics. This is not a criticism of IBM or IonQ's own tools. Every one of these bugs lived in our own code, sitting on top of both platforms' real, correct SDKs. It's a caution about how easy it is to mistake "my code has a bug" for "this hardware has a limitation," especially when the result still comes back looking plausible.

The second way is the one the physics finding is about: a real hardware effect that only looks stable because you only looked once. Our first `single`-vs-`ring` comparison was clean, confident-looking, and wrong in its conclusion, not because anything was broken, but because a single run of a noisy process is not the same thing as a measurement of that process. The fix for the first kind of lie is better software. The fix for the second kind is more boring and more important: repeat the measurement before you trust it, especially before you publish a claim about which vendor is "better" at something.

Those two lessons point at the same practical conclusion for anyone doing this kind of comparison. Build the checking in as infrastructure you run by default, not as something you remember to do after a result looks surprising. Case study 6 exists precisely because the same discipline that catches real bugs also has to be trusted to correctly say "this one's fine," or it isn't worth using at all. And publish enough that someone else can independently re-run your exact circuit against the same job IDs on the vendor's own systems. That's a stronger form of evidence than any figure or screenshot we could include here, because it doesn't depend on trusting us.

We also want to be honest about the size of the claim this actually supports, and it's smaller than our first draft of this section claimed. This is one search circuit, with one specific connectivity pattern, on two specific real devices, on one specific night. It is not evidence that IonQ hardware is generally less stable than IBM's, or that either platform is "better" in any broad sense. But it's narrower than that even within this one result: IonQ's two runs were six

days apart with a shot count change in between; IBM's two runs were 37 minutes apart with nothing changed. We measured a real, statistically significant change in IonQ's result over its (longer, shot-count-changed) interval, and no significant change in IBM's result over its (much shorter, unchanged) interval. That is a real, repeat-confirmed pair of measurements. It is not, on its own, proof that IonQ is the less stable vendor, because we never measured IonQ over a matched 37-minute window or IBM over a matched six-day window. What we're confident in, no further than the data actually goes: these two specific deltas are real and were not shot noise. Which one of them reflects vendor behavior and which reflects elapsed time is the open question a matched repeat would settle.

Conclusion

We set out to compare IBM and IonQ hardware honestly, and the first thing that took was admitting how easy it is to do that dishonestly by accident. Before we could trust any real comparison, we had to build a tool that checks whether the circuit that actually ran on hardware is the circuit we meant to run, and that tool immediately found five real bugs in our own work, several of which would have silently produced a wrong or misleading result if we hadn't caught them first. It also, once, correctly found nothing, which mattered just as much.

Only once those checks were in place did we trust the actual physics result: a real, repeat-confirmed, statistically significant change in IonQ's result on a five-qubit ring circuit across a six-day gap, against no significant change in IBM's result across a 37-minute gap — a finding that only exists because we repeated the measurement instead of trusting a single clean-looking run. The first version of that comparison told a tidier, more publishable-sounding story, and it was wrong. We're deliberately not rounding this up to "IonQ is the less stable vendor": the two gaps being compared differ by three orders of magnitude, so vendor identity and elapsed time are confounded here, and only a matched repeat can separate them.

What's left open, honestly: we don't yet know what's driving IonQ's change, only that it isn't fully explained by the calibration drift we could measure, and we don't yet know how much of it is "IonQ" versus "six days." The real next step is the matched design this result is missing: a same-night IonQ repeat (to see whether IonQ holds steady over 37 minutes the way IBM did) and a multi-day IBM repeat (to see whether IBM drifts over six days the way IonQ did). Only that pair of experiments can separate "vendor" from "elapsed time" as the actual cause. A redesigned, triangle-free version of our third circuit would let us extend this comparison to higher connectivity on IBM without the routing overhead that ruled it out here. And the angle-error question we resolved on the free simulator hasn't yet been run for real. All three are real next steps, not gaps we're glossing over.

If there's one thing worth taking from this beyond the specific numbers, it's that the tool and the finding depend on each other. Neither one means much without the other: a clean result from an unverified pipeline isn't trustworthy, and a verification tool with nothing real to verify isn't useful. We built both together, on purpose, and that's the actual contribution here.

Data availability

The verification tool and the main MCP server behind this work are public: github.com/Lokesh-2025/quantum-verifier and github.com/Lokesh-2025/quantum-hardware-mcp. The specific circuits and raw job data behind this paper's physics result are published as a dedicated, minimal public data release, separate from the authors' broader private research repositories: github.com/Lokesh-2025/ionq-ibm-cross-vendor-data, archived on Zenodo for a permanent, citable record at DOI 10.5281/zenodo.23543567. Every job ID listed in Table 5 below is independently checkable against IBM's and IonQ's own job history by anyone with API access to those platforms, which does not depend on repository access at all and is the strongest form of verification available here.

Table 5. Real job IDs behind the Physics finding section.

Run	Provider	Circuit(s)	Job ID
Run 1 (2026-08-13, 2048 shots)	IonQ	single, ring, degree3 (batched)	`019ff976-2728-7726-8839-9109c9122b98`
Run 2 / repeat (2026-08-19, 4096 shots)	IonQ	single, ring (batched)	`01a0184e-9b2e-707e-a336-d6c7d4c4ce7b`
Run 1 (2026-08-19, 4096 shots)	IBM (`ibm_fez`)	single	`da2ikiuaa69c739hi5sg`
Run 1 (2026-08-19, 4096 shots)	IBM (`ibm_fez`)	ring	`da2iklmaa69c739hi600`
Run 2 / repeat (2026-08-19, 4096 shots)	IBM (`ibm_fez`)	single	`da2j5se1vhnc73fip530`

Run	Provider	Circuit(s)	Job ID
Run 2 / repeat (2026-08-19, 4096 shots)	IBM (`ibm_fez`)	ring	`da2j5uuaa69c739hipkg`

Acknowledgments

Thanks to Vadim Karpusenko and the IonQ Research Credits program for the real hardware access and credits that made the IonQ side of this comparison possible.

References

1. Qiskit contributors. “Qiskit: An Open-Source Framework for Quantum Computing.” *Zenodo*, 2023. DOI: 10.5281/zenodo.2573505.
2. Wilson, E. B. “Probable Inference, the Law of Succession, and Statistical Inference.” *Journal of the American Statistical Association*, vol. 22, no. 158 (June 1927), pp. 209–212. DOI: 10.1080/01621459.1927.10502953. (Origin of the Wilson score confidence interval used in our ground-truth significance test.)
3. Schuirmann, D. J. “A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability.” *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6 (1987), pp. 657–680. DOI: 10.1007/BF01068419. (Origin of the TOST equivalence-testing method our verification pipeline adapts.)
4. IBM Quantum. Hardware documentation and heavy-hex coupling-map reference, https://quantum.ibm.com.
5. IonQ. Trapped-ion hardware and native gate-set documentation, https://ionq.com/docs.
6. Pullakandam, L., Woehr, J. `singmasters-conjecture`: Grover’s search and LNAA for Singmaster’s Conjecture on IBM Quantum hardware. github.com/Lokesh-2025/singmasters-conjecture (public repository; source of the 27.78x/178.8x results cited in the Introduction).